

Deep Reinforcement Learning for Bomberman

Eric Flämig, Mark Salzmann, Jesper Eggers

September 21, 2026

Abstract

Bomberman combines navigation, delayed rewards, and decisions that can place the agent itself in danger. This report discusses the two agents we developed parallelly, both relying on the Double Deep Q-Network (Double DQN) learning algorithm.

1 Theoretical Background and Agent Architecture

1.1 Bomberman as a reinforcement learning problem

At each time step, the agent observes the game, chooses an action, and receives feedback through game events. Its action set is

$$\mathcal{A} = \{\text{UP}, \text{RIGHT}, \text{DOWN}, \text{LEFT}, \text{WAIT}, \text{BOMB}\}.$$

An episode corresponds to a round of play from the agent’s perspective and ends when it dies or the round terminates. The configured board has 17×17 tiles, with a maximum of four agents and 400 steps per round. The aim of the game is to collect coins and eliminate opponents. Coins yield points immediately, whereas placing a bomb can produce a benefit only after its timer expires. The agent must therefore consider future consequences rather than only the next reward.

The usual theoretical model is a Markov decision chain with states, actions, transition probabilities, rewards, and discount factor γ . The state of the game s_t includes the positions of all agents, bomb timers, and revealed coins, as well as any other relevant game information needed to make decisions. To make optimal decisions, the next action should be chosen based on the expected future rewards, considering both immediate and delayed consequences, captured by the action-value function defined as

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right]. \quad (1)$$

Here, r_{t+1} is the learning reward, T is the episode’s terminal time, and π is the policy. A larger discount factor gives delayed outcomes greater weight. For example, a useful bomb placement may deserve a high value even if the agent must first move away and wait for the explosion.

1.2 From Q-learning to deep Q-learning

A deep Q-network (DQN) uses a neural network to approximate the action-value function, allowing it to generalize across similar states and handle large state spaces. For a given input state s the network outputs a vector of action values $Q(s, a, \theta)$ for all possible actions $a \in \mathcal{A}$.

The learning process updates the network parameters θ using stochastic gradient descent

$$\theta_{t+1} = \theta_t + \alpha \left(Y_t^Q - Q(S_t, A_t, \theta_t) \right) \nabla_{\theta} Q(S_t, A_t, \theta_t), \quad (2)$$

where α is the learning rate and Y_t^Q is the target value for the Q-network at time step t defined as

$$Y_t^Q = r_{t+1} + \gamma; \max_a Q(S_{t+1}, a, \theta_t). \quad (3)$$

Previous steps get stored in the *experience replay* buffer, which allows the network to sample minibatches of past transitions for training, reducing the correlation between consecutive updates. Furthermore the target network with parameters θ^- is used to compute the target values, and it is updated every τ steps from the online network’s parameters such that $\theta^- = \theta_t$. The target used in the update then becomes

$$Y_t^Q = r_{t+1} + \gamma \max_a Q(S_{t+1}, a, \theta^-), \quad (4)$$

where θ^- are the parameters of the target network.

1.3 Double DQN: separating selection and evaluation

The maximum in Equation (4) selects and evaluates an action using the same estimates. The issue with this method is that is prone to overestimation of action values, leading to bias in the learned Q-values. Van Hasselt et al. show that this overestimation also occurs with deep networks and propose Double DQN to reduce it [2].

They therefore propose the Double DQN approach, which uses the online network with parameters θ to select the next action and the target network with parameters θ' to evaluate that action. These two networks are then updated independently, by assigning the change to either of the networks randomly during training. Important is that the changing the roles of the online and target networks is done in a symmetric manner. The double DQN target is defined as

$$Y_t^{\text{DoubleDQN}} = r_{t+1} + \gamma Q\left(s_{t+1}, \arg \max_a Q(s_{t+1}, a, \theta_t); \theta'\right), \quad (5)$$

and note that action selection in the $\arg \max$ still uses the online parameter θ_t . The best action of the greedy policy is still estimated using the current values, but the policy networks θ' is used to provide an unbiased evaluation of that action’s value.

In Bomberman, a relevant motivation is to avoid overvaluing risky actions whose apparent payoff is based on inaccurate predictions. The double DQN approach is beneficial in this context, as it helps the agent get more accurate value estimates for risky actions, leading to safer and more effective decision-making.

1.4 Input representation and feature engineering

The function `state_to_features` converts the observation into seven spatial maps and ten scalar values. The board input has shape $7 \times 17 \times 17$ and uses game information directly rather than rendered screen pixels. Table 1 summarizes its channels.

From these standard features, one can construct further informative representations, such as combining bomb locations with the explosion map to a combined **danger map**.

The scalar input consists of the number of rounds, and steps played so far in the game. Additionally it is useful for training to provide information about the agent’s bomb availability, the number of

Channel	Encoding
Walls	Binary map of indestructible walls.
Crates	Binary map of destructible crates.
Coins	Binary map of currently visible coins.
Bombs	Binary map of placed bomb locations.
Explosion map	Integer map of active explosions; the value represents the remaining duration.
Own position	Onehot encoding of the agent’s tile
Opponents	Onehot encoding of each opponent’s tile.

Table 1: Spatial features implemented in `callbacks.py`.

remaining coins, the number of remaining opponents, the current step, and the previous action. Therefore we defined

Channel	Encoding
Bomb availability	Binary value indicating if the agent can place a bomb.
Number of coins normalised	Float32 value representing the number of all collectable coins divided by the total number of coins.
Number of opponents	Integer value representing the number of opponents.
Current step	Integer value representing the current step in the game.
Previous action	One-hot encoding of the previous action taken by the agent.
Own position	Onehot encoding of the agent’s tile
Opponents	Onehot encoding of each opponent’s tile.

Table 2: Scalar features implemented in `callbacks.py`.

By providing the agent with absolute information about the game state, such as the number of remaining coins, opponents, and the current step, the network can make more informed decisions.

1.5 Exploration with an epsilon-greedy policy

The agent implements an ϵ -greedy policy. Let $\mathcal{A}_m(s)$ be the actions retained by its mask. During training, the action is sampled as

$$a_t = \begin{cases} \text{a uniform random action from } \mathcal{A}_m(s_t), & \text{with probability } \epsilon_t, \\ \arg \max_{a \in \mathcal{A}_m(s_t)} Q_\theta(s_t, a), & \text{with probability } 1 - \epsilon_t. \end{cases} \quad (6)$$

Exploration supplies experience with actions that the current network may undervalue, whereas exploitation uses what has already been learned. Both branches respect the mask. The random branch can also select the greedy action, so ϵ_t is the probability of random selection, not necessarily the probability of taking a non-greedy action.

In order to provide sufficient exploration early in training, but gradually reduce it as the agent learns, we use an exponentially decaying epsilon

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\text{start}} - \epsilon_{\min}) \cdot \exp\left(-\frac{t}{T_{\text{decay}}}\right). \quad (7)$$

It is beneficial to choose $T_{\text{decay}} = \frac{T_{\text{Half-life}}}{\ln 2}$.

During the whole training process, the agent is faced with different training epochs. This introduces varying scenarios, obstacles, and opponents. To provide adequate exploration, it is a good trick to reset epsilon to a middle value at the beginning of each new training epoch, as discussed in 1.6.2.

1.6 Ruehl based agent

1.6.1 Neural network architecture

Given the combination of spatial and scalar features, it is reasonable to use a network architecture that processes these two types of inputs separately before combining them for decision making. The spatial branch contains four convolutional layers with ReLU activations. Dilated convolutions expand the area visible to a filter without increasing its number of weights, while the third convolution reduces the spatial resolution. Table 3 gives the dimensions derived from `Networks.py` with the batch dimension is omitted.

Component	Operation	Output shape
Board input	Seven spatial channels	$7 \times 17 \times 17$
Convolution 1	3×3 , 32 filters, stride 1, padding 1	$32 \times 17 \times 17$
Convolution 2	3×3 , 32 filters, dilation 2, padding 2	$32 \times 17 \times 17$
Convolution 3	3×3 , 32 filters, stride 2, padding 1	$32 \times 9 \times 9$
Convolution 4	3×3 , 32 filters, dilation 2, padding 2	$32 \times 9 \times 9$
Flatten	Flatten the board features	2592
Scalar branch	Linear $10 \rightarrow 32$, ReLU	32
Concatenation	Join spatial and scalar features	2624
Shared layers	Linear $2624 \rightarrow 128 \rightarrow 128$, ReLU after each	128
Value head	Linear $128 \rightarrow 1$	1
Advantage head	Linear $128 \rightarrow 6$	6

Table 3: Architecture of the RUEHL network for the configured board size.

Each output of the last convolution has a theoretical receptive field of 17×17 input tiles, including padding near the boundary. The flattened features subsequently connect to shared fully connected layers.

Complementing the double DQN strategy, one can employ a dueling network architecture to separately estimate state values and action advantages, which can lead to more stable learning and improved policy evaluation. In the dueling DQN architecture, the action-value function is not represented directly by a single network output. Instead, the network decomposes the estimate of $Q(s, a)$ into two components: a state-value function $V_\theta(s)$ and an action-dependent advantage function $A_\theta(s, a)$ [3]. The state-value function represents the overall quality of being in state s , independently of which action is selected, whereas the advantage function describes the relative benefit of choosing a particular action a in that state.

A naive decomposition of the form

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a)$$

is not identifiable. In particular, an arbitrary constant c can be added to the value function and subtracted from all advantage values without changing Q_θ :

$$V_\theta(s) + c + (A_\theta(s, a) - c) = V_\theta(s) + A_\theta(s, a).$$

Consequently, the network would have no unique way of determining which part of the predicted action value should be represented by V_θ and which part should be represented by A_θ .

To remove this ambiguity, the dueling architecture combines the two streams by subtracting the

mean advantage over all available actions,

$$Q_\theta(s, a) = V_\theta(s) + A_\theta(s, a) - \frac{1}{|\mathcal{A}|} \sum_{b \in \mathcal{A}} A_\theta(s, b). \quad (8)$$

This centers the effective advantage values around zero. An important consequence of this parameterization is obtained by averaging $Q_\theta(s, a)$ over all actions:

$$\frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} Q_\theta(s, a) = V_\theta(s).$$

Thus, under this aggregation rule, the value stream represents the mean predicted action value in a state, while the centered advantage stream determines how the individual action values differ from this common baseline. In particular, because both $V_\theta(s)$ and $\bar{A}_\theta(s)$ are independent of the selected action,

$$\arg \max_a Q_\theta(s, a) = \arg \max_a A_\theta(s, a).$$

Hence, the advantage stream determines the relative ranking of actions, whereas the value stream controls the overall level of their Q-values.

The motivation for this decomposition is that in many states the precise choice of action has only a small effect on the expected return. A conventional DQN must nevertheless estimate a separate $Q(s, a)$ for every action. The dueling architecture allows the network to learn features that characterize whether a state is generally favorable or unfavorable through $V_\theta(s)$, while using the advantage stream only to represent differences between actions. This can make value estimation more efficient in states where several actions have similar consequences [3].

The two quantities are not trained using separate value and advantage targets. Instead, they are combined according to Eq. (8) to obtain $Q_\theta(s, a)$, and the usual temporal-difference loss is applied to this resulting Q-value. Gradients then propagate through the aggregation operation into both the value and advantage streams. The decomposition is therefore an architectural parameterization of the Q-function rather than the introduction of two independently supervised learning objectives.

1.6.2 Training curriculum

The training progression considered in this project starts with navigation, then introduces bomb use, and finally adds opponents. This curriculum is motivated by the difficulty of discovering all three behaviors at once: an inexperienced agent can otherwise die before collecting useful examples of delayed rewards. Table 4 links the progression to the available scenarios.

Stage 1: coin collection. In the `coin-heaven` scenario, visible coins give a relatively direct objective: move through the maze and collect them efficiently. The agent can learn spatial relationships between its position, walls, and rewards. For this initial stage of training, we started the epsilon greedy strategy with $\epsilon = 1$, ensuring maximum exploration. Via reward shaping, the agent receives immediate feedback for collecting coins, and negative feedback for undesirable actions, guiding it towards efficient coin collection.

Scenario	Crate density	Coins	Role in the curriculum
<code>coin-heaven</code>	0	50	Learn movement and collection; the initial stage is intended to run without opponents.
<code>loot-crate</code>	0.75	50	Learn to open routes, reveal coins, and escape after placing bombs.
<code>classic</code>	0.75	9	Combine collection and bomb use with competition against opponents.

Table 4: Configured scenarios and their intended training roles. Crate density is the placement probability before walls and starting areas are cleared. Opponents are configured separately from the scenario.

Stage 2: crate destruction. The `loot-crate` scenario adds destructible obstacles and places coins preferentially underneath them. Successful collection can now require a sequence of actions: approach a crate, place a bomb, reach safety, and return to collect a revealed coin. This stage introduces delayed credit assignment and the conflict between making progress and avoiding self-destruction. Initializing from the pretrained Network on `coin-heaven`, we continued training with substantial epsilon reheating $\epsilon = 0.85$ to encourage exploration in the presence of destructible crates. For this stage, a different reward profile is used, emphasizing successful crate destruction and safe bomb placement, while still rewarding coin collection and very importantly, penalises suicidal actions.

Stage 3: opponents and competitive play. The main goal for this stage is to adapt to less placed coins, while dealing with dynamic opponents. A progression from simpler opponents to stronger ones would let the agent adapt these skills gradually. For this purpose, we iteratively improved the model, by transferring the pretrained network from `loot-crate` to `classic` with slight epsilon reheating $\epsilon = 0.20$, then gradually increased the difficulty of opponents.

1.6.3 Action masking and hand-designed guidance

The mask is an important part of an effective policy. It first rejects movement into walls, crates, bombs, or opponents. It then filters movement and waiting on active explosions or blast tiles of bombs whose timers are at most one. Bomb placement is allowed only when a bomb is available, its blast could hit a crate or opponent, and a static breadth-first search finds a tile outside its blast within the configured bomb timer.

If no action survives the danger filter, the code falls back to the earlier legal-action mask. If that is also empty, the model chooses `WAIT`. This is therefore a heuristic filter, not a guarantee of survival. Its static escape search does not predict moving opponents or jointly simulate other bombs over time.

This guidance reduces the exploration burden, but it also contributes to the observed behavior independently of learned Q-values. Results should therefore describe the complete network-plus-mask policy. The Double DQN update in `train.py` also applies the next-state action mask before the online network selects its maximizing action. The target network then evaluates that selected action.

1.6.4 Reward shaping

The main goal of the agent is to maximize game score at the end of the game, by collecting coins, eliminating opponents, given by

$$S = N_{\text{coins}} + 5N_{\text{opponents killed}}. \quad (9)$$

However, starting from these sparse and delayed objectives alone makes it difficult for the agent to learn effective strategies. Reward shaping addresses this problem, by giving intermediate rewards and punishments that guide the agent’s behavior before the final game outcome is known. For the `ruehl_based_agent`, we rejected anisotropic rewards, meaning rewards that vary depending on the direction or context of the action. Combined with the CNN architecture, this design choice ensures that the agent exploits the symmetry of the game environment. Furthermore, we only introduce additional shaping rewards that are deductible from the input game state, e.g. rewarding the survival of own placed bombs or penalizing the agent for being in the blast radius of an active explosion. Furthermore, one can differentiate between main rewards, connecting to significant game events, and shaping rewards, which serve as fine tuning of the agent’s behavior. The shaping rewards are an order of magnitude smaller than the main rewards.

Main rewards The main contribution, stored under `major`, assigns values to selected game events. Collecting a coin provides a baseline reward, typically 1 point, in accordance with the actual game score. Other rewards will be scaled relative to this baseline. A separate Self-destruction penalty is useful in competitive play. This expresses the trade-off between gaining points and retaining the opportunity to act. Penalizing only death would encourage excessive caution, meanwhile rewarding only attacks would provide insufficient feedback about their danger. Their combination is intended to encourage productive risks with an escape plan.

Shaping penalties The separate shaping terms discourage waiting, entering an active explosion, and, in the competitive profile, immediately reversing movement direction. Their purpose is to discourage inactivity and repeated back-and-forth motion without making these actions impossible. Waiting for an explosion to clear or reversing direction to escape can be necessary, so these penalties should remain small relative to important outcomes.

1.6.5 Task-specific reward profiles

We identified three substantially different challenges, for which we designed corresponding reward profiles: `coin`, `loot_exploration`, and `killer`. Determining the appropriate distribution of rewards was a challenging task that required long trial and error processes. The current values are summarized in Table 5.

Coin collection. The `coin` profile rewards collection and discourages waiting through both a shaping penalty and a small step cost. It gives no direct incentive for crate destruction or bomb placement, focusing the initial task on navigation. Opponent-kill rewards are inherited from the shared reward table but have no effect in a scenario without opponents.

Crate exploration. The `loot_exploration` profile rewards destroying crates, revealing coins, and placing bombs whose blast reaches a crate. These intermediate rewards provide feedback before the agent eventually collects a coin. Removing the step cost and reducing the waiting penalty allows more time for bomb placement and escape sequences, while the self-destruction penalty retains an incentive to survive.

Event or parameter	coin	loot_exploration	killer
Coin collected	1.0	1.0	1.0
Coin revealed	0.1	0.5	0.5
Crate destroyed	0	0.35	0.15
Bomb placed to hit a crate	0	0.05	0.02
Bomb placed to threaten an opponent	0	0	0.75
Opponent killed	6.5	6.5	6.0
Killed by another agent	-1.0	-1.0	-2.0
Self-destruction	-2.0	-2.0	-6.0
Bomb exploded	0	0	0.15
Round survived	0	0	0.75
Waiting (shaping)	-0.08	-0.01	-0.05
Danger (shaping)	-0.02	-0.04	-0.10
Reversing direction (shaping)	0	0	-0.03
Shaping limit c	0.05	0.05	0.15
Step cost λ	0.008	0	0.002

Table 5: Reward profiles in `train.py`. Event values are summed per transition; only the combined shaping contribution is clipped. A zero denotes an event with no assigned reward in that profile.

Competitive play. The `killer` profile emphasizes successful attacks and survival. A bomb that currently threatens an opponent earns an immediate reward of 0.75, while a confirmed kill earns 6.0. Self-destruction costs 6.0, substantially more than death caused by another agent. Smaller crate rewards preserve an incentive to open routes, and surviving the round earns 0.75. The bomb-placement event checks the current blast geometry, not whether the opponent will actually be hit. If both opponents and crates are in range, only the opponent-targeting placement event is added.

1.6.6 Optimization and training state

The training loop stores transitions in a replay buffer of fixed size, and samples minibatches uniformly without replacement, starting after the warm-up period. It implements the Double DQN target from Equation (5): the masked online network selects the next action, and the target network evaluates it. For optimization, the `ruehl_based_agent` uses AdamW with a smooth L1 loss, and clips the gradient norm to 5.0. The target network receives a hard copy of the online weights every `TARGET_UPDATE` gradient updates.

The replay capacity, batch size, discount factor, learning rate, warm-up, update intervals, and evaluation budget are read from `Hyperparams.prm`.

1.6.7 Evaluation and model selection

After `EVAL EVERY TRAINING ROUNDS` training rounds, the agent plays `EVAL ROUNDS` greedy evaluation rounds with $\epsilon = 0$. During these rounds, no transitions are added to the replay buffer and no optimization or exploration decay steps occur. Periodic evaluation ensures that model selection is based on up-to-date performance rather than outdated checkpoints, especially for DQN agents [1]. For coin collection and crate exploration, the selection is based on the actual game score rather than accumulated shaping rewards. In competitive play, the selection score is extended by penalising self-destruction and favoring eliminating opponents.

2 Results and Discussion

2.1 Ruehl based agent

Table 6 summarizes the best greedy evaluation observed during each curriculum stage. Each reported score is the mean over the 20 evaluation games used for checkpoint selection, whereas the first row gives the number of training games assigned to the stage.

Metric	Coin collection	Loot crate	Peaceful killer	Classic killer	Classic self-play	Classic hard opponents
Training games	30,000	60,000	120,000	30,000	30,000	18,500
Mean game score	50.00	44.90	6.45	7.95	7.10	3.95
Mean coins collected	50.00	44.90	3.20	4.20	2.60	1.95
Mean opponents killed	0.00	0.00	0.65	0.75	0.90	0.40
Completion rate (%)	100	20	0	0	0	0
Suicide rate (%)	0	5	20	5	10	10
Mean crates destroyed	0.00	43.40	44.05	44.60	36.35	29.10

Table 6: Best-model evaluation metrics across the six training stages. Values are taken from the corresponding `Hyperparams.prm` snapshots.

Figure 1 depicts how the selected best-model score evolves with training. The evaluations take place every 200 games, and the current best model is saved if its score improves. The score converges fairly quickly in each stage, approximately within the first 5,000 games, and then plateaus. In the `classic killer` stage, the score has jumped at 28,000 games, suggesting that improvement continues beyond the initial plateau. This slow progression is characteristic of DQN training: periodic target-network synchronization changes the temporal-difference targets and therefore the loss landscape, while evaluation only measures the resulting policy. The results after a long enough training period are summarized in Table 6.

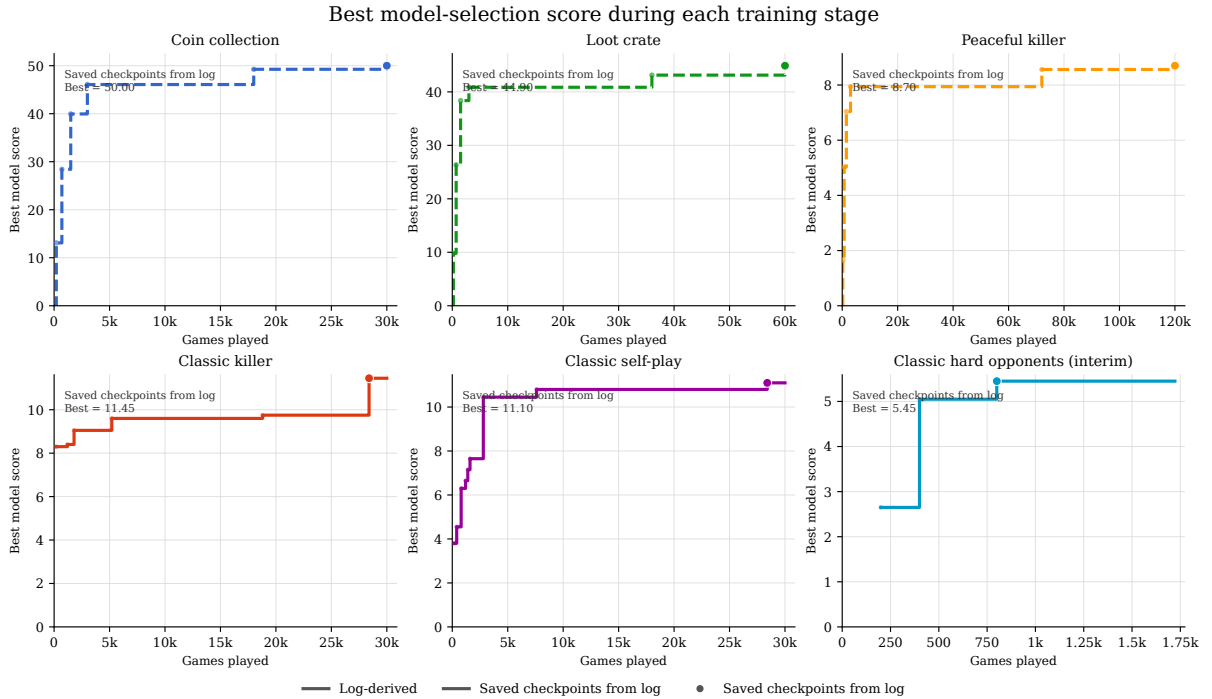


Figure 1: Best model-selection score as a function of games played in each training stage. The evaluations take place every 200 games, and the current best model is saved if its score improves. The scores of the best models at each checkpoint are reported in Table 6.

2.2 Conclusion

3 Conclusion

We developed and evaluated a different agents based on Double DQN, a dueling network architecture, engineered spatial and scalar features, action masking, and a staged training curriculum. The curriculum successfully decomposed the task into progressively harder objectives: the agent learned coin collection perfectly in `coin-heaven` and acquired effective crate-destruction behavior in `loot-crate`. In the competitive stages, it also learned to destroy crates, collect coins, and occasionally eliminate opponents while keeping self-destruction rates comparatively low.

In the case of the `ruehl_base_agent`, the agent struggled to achieve consistent success against hard opponents, highlighting the gap between performance in simplified training environments and competitive `classic` games. In particular, the best model against hard opponents achieved a mean score of 3.95 over 20 evaluation games, with a mean of 0.40 opponent eliminations and no completed rounds. Thus, the agent learned meaningful bomb-use and survival-related behavior, but is unable to overcome the enemies counter-strategies consistently.

Possible next steps include evaluation over longer training periods, more systematic hyperparameter tuning, and training against a diverse population of opponents. These extensions would clarify which design choices contribute most strongly to performance and may improve robustness in the full competitive setting.

References

- [1] Melrose Roderick, James MacGlashan, and Stefanie Tellex. Implementing the deep Q-network. *arXiv preprint arXiv:1711.07478*, 2017. URL <https://arxiv.org/abs/1711.07478>. Version 1. Local source: 1711.07478v1.pdf.
- [2] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. *arXiv preprint arXiv:1509.06461*, 2015. URL <https://arxiv.org/abs/1509.06461>. Version 3. Local source: 1509.06461v3.pdf.
- [3] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003. PMLR, 2016. URL <https://proceedings.mlr.press/v48/wangf16.html>.